

Código mandanga NavMesh&NavMesh Agent.

Movimiento autónomo entre obstacles con limitación por distancia. La idea es sacar los puntos en el mapa a tener en cuenta, la ruta, evitando los obstacles y comprobar si está capacitado para recorrerla teniendo en cuenta su valor distancia. Además proyectaremos la zona de acción con LineRenderer así como gestionar la velocidad del agente para lanzar según qué animación.

Versión unificada try hard:

```
using UnityEngine;
using UnityEngine.AI;

[RequireComponent(typeof(NavMeshAgent), typeof(LineRenderer), typeof(Animator))]
public class PersonajeCompleto : MonoBehaviour
{
    [Header("Configuración de Movimiento")]
    public float distanciaMaximaMovimiento = 5f;
    public LayerMask capaSuelo;

    [Header("Visualización de Rango")]
    public int segmentosCirculo = 60;
    public float alturaLinea = 0.05f;
    public Color colorLinea = Color.cyan;
    public float grosorLinea = 0.1f;

    [Header("Animación")]
    public string parametroVelocidad = "Velocidad";
    public float suavizadoAnimacion = 0.1f;

    private NavMeshAgent agente;
    private LineRenderer linea;
    private Animator animator;
    private NavMeshPath rutaCache;
    private float velocidadSuavizada;

    void Start()
    {
        // Inicializar componentes
        agente = GetComponent<NavMeshAgent>();
        linea = GetComponent<LineRenderer>();
        animator = GetComponent<Animator>();
        rutaCache = new NavMeshPath();

        // Configurar visualización de rango
        ConfigurarLineaRender();
        DibujarCirculoRango();
    }

    void Update()
    {
        ProcesarInputMovimiento();
        ActualizarAnimacion();
    }
}
```

```

void ProcesarInputMovimiento()
{
    if (Input.GetMouseButtonDown(0))
    {
        Ray ray = Camera.main.ScreenPointToRay(Input.mousePosition);

        if (Physics.Raycast(ray, out RaycastHit hit, 100f, capaSuelo))
        {
            Vector3 destinoPropuesto = hit.point;

            if (agente.CalculatePath(destinoPropuesto, rutaCache))
            {
                float distanciaRuta = CalcularDistanciaRuta(rutaCache);

                if (distanciaRuta <= distanciaMaximaMovimiento)
                {
                    agente.SetPath(rutaCache);
                }
                else
                {
                    Debug.Log("Destino fuera del rango permitido");
                }
            }
        }
    }
}

void ActualizarAnimacion()
{
    float velocidadActual = agente.velocity.magnitude;
    velocidadSuavizada = Mathf.Lerp(velocidadSuavizada, velocidadActual, suavizadoAnimacion *
Time.deltaTime);
    animator.SetFloat(parametroVelocidad, velocidadSuavizada);
}

void ConfigurarLineaRenderer()
{
    linea.loop = true;
    linea.widthMultiplier = grosorLinea;
    linea.useWorldSpace = false;
    linea.positionCount = segmentosCirculo;

    // Crear material simple si no hay uno asignado
    if (linea.material == null)
    {
        linea.material = new Material(Shader.Find("Unlit/Color"));
        linea.material.color = colorLinea;
    }
}

void DibujarCirculoRango()
{
    float angulo = 0f;
    float incrementoAngulo = 360f / segmentosCirculo;

    for (int i = 0; i < segmentosCirculo; i++)
    {

```

```

        float x = Mathf.Cos(Mathf.Deg2Rad * angulo) * distanciaMaximaMovimiento;
        float z = Mathf.Sin(Mathf.Deg2Rad * angulo) * distanciaMaximaMovimiento;
        linea.SetPosition(i, new Vector3(x, alturaLinea, z));
        angulo += incrementoAngulo;
    }
}

float CalcularDistanciaRuta(NavMeshPath ruta)
{
    float distanciaTotal = 0f;
    for (int i = 1; i < ruta.corners.Length; i++)
    {
        distanciaTotal += Vector3.Distance(ruta.corners[i-1], ruta.corners[i]);
    }
    return distanciaTotal;
}

void OnValidate()
{
    // Actualizo el círculo en el editor cuando se cambian los valores
    if (linea != null && segmentosCirculo > 3)
    {
        ConfigurarLineaRenderer();
        DibujarCirculoRango();
    }
}
}

```

Traviesas Características	¿Cómo usarlo?
- Movimiento con límite de distancia - Detección de clics en terreno - Visualización del área de movimiento - Animaciones suavizadas - Actualización en tiempo real en el editor (OnValidate)	-Agrega este script a tu personaje -Asegúrate que tenga los componentes requeridos: - NavMeshAgent - LineRenderer - Animator

Sección troceada de la mandanga(opción B)

SOLO MOVIMIENTO:

```

using UnityEngine;
using UnityEngine.AI;

[RequireComponent(typeof(NavMeshAgent))]
public class ControlMovimientoPorClic : MonoBehaviour
{
    public float distanciaMaximaMovimiento = 5f;
    public LayerMask capaSuelo;
    private NavMeshAgent agente;
}

```

```

void Start()
{
    agente = GetComponent<NavMeshAgent>();
}

void Update()
{
    if (Input.GetMouseButtonDown(0))
    {
        Ray ray = Camera.main.ScreenPointToRay(Input.mousePosition);

        if (Physics.Raycast(ray, out RaycastHit hit, 100f, capaSuelo))
        {
            Vector3 destinoPropuesto = hit.point;

            // Calcular si se puede llegar hasta ahí por el camino de la NavMesh
            NavMeshPath ruta = new NavMeshPath();
            if (agente.CalculatePath(destinoPropuesto, ruta))
            {
                float longitudRuta = CalcularLongitudRuta(ruta);
                if (longitudRuta <= distanciaMaximaMovimiento)
                {
                    agente.SetPath(ruta);
                }
                else
                {
                    Debug.Log("¡Destino demasiado lejos!");
                }
            }
        }
    }
}

float CalcularLongitudRuta(NavMeshPath ruta)
{
    float total = 0f;
    for (int i = 1; i < ruta.corners.Length; i++)
    {
        total += Vector3.Distance(ruta.corners[i - 1], ruta.corners[i]);
    }
    return total;
}

```

SOLO line renderer

using UnityEngine;

```

[RequireComponent(typeof(LineRenderer))]
public class CirculoRangoMovimiento : MonoBehaviour
{
    public float radio = 5f;

```

```

public int segmentos = 60;
private LineRenderer linea;

void Start()
{
    linea = GetComponent<LineRenderer>();
    linea.loop = true;
    linea.widthMultiplier = 0.1f;
    linea.useWorldSpace = false;
    linea.material = new Material(Shader.Find("Unlit/Color"));
    linea.material.color = Color.cyan;

    linea.positionCount = segmentos;
    DibujarCirculo();
}

void DibujarCirculo()
{
    float angulo = 0f;
    for (int i = 0; i < segmentos; i++)
    {
        float x = Mathf.Cos(Mathf.Deg2Rad * angulo) * radio;
        float z = Mathf.Sin(Mathf.Deg2Rad * angulo) * radio;
        linea.SetPosition(i, new Vector3(x, 0.05f, z));
        angulo += 360f / segmentos;
    }
}
}

```

SOLO BLEND TREE

```

using UnityEngine;
using UnityEngine.AI;

public class ControlAnimacionMovimiento : MonoBehaviour
{
    public Animator animator;
    private NavMeshAgent agente;

    void Start()
    {
        agente = GetComponent<NavMeshAgent>();
    }

    void Update()
    {
        float velocidad = agente.velocity.magnitude;
        animator.SetFloat("Velocidad", velocidad);
    }
}

```

Abrimos el animator y creamos un nuevo Blend Tree. Creamos el parámetro Float de nombre velocidad.

El Blend Tree será de 1D y tendrá en cuenta el parámetro velocidad.

Añadimos dos animaciones: Idle con valor 0 y Correr con valor 3(o lo que sea nuestra velocidad máxima)

Ojo, notas!!

Pintar el borde del combate!! Creamos un nuevo objeto vacío con nombre limiteCombate(fuera tildes) y le añadimos el LineRenderer. Además tendremos este script bien radiactivo para marcar los límites de la zona:

```
using UnityEngine;

[RequireComponent(typeof(LineRenderer))]
public class LimiteCombate : MonoBehaviour
{
    public float minX = -5f;
    public float maxX = 5f;
    public float minZ = -5f;
    public float maxZ = 5f;
    // Altura donde dibujar la línea(ojo, mandanga)
    public float altura = 0.05f;
    private void Start()
    {
        LineRenderer lr = GetComponent<LineRenderer>();
        lr.loop = true;
        lr.positionCount = 5;

        Vector3[] puntos = new Vector3[5]
        {
            new Vector3(minX, altura, minZ),
            new Vector3(maxX, altura, minZ),
            new Vector3(maxX, altura, maxZ),
            new Vector3(minX, altura, maxZ),
            new Vector3(minX, altura, minZ)
        };

        lr.SetPositions(puntos);

        // Ajustes visuales
        lr.widthMultiplier = 0.1f;
        lr.useWorldSpace = true;
    }
}
```

Ahora crearemos un material brillante emisoro para URP bajo el Shader: URP/Unlit/Color. Aumentamos la intensidad de la emisión y asignamos el material al LineRenderer. Opcionalmente podemos activar "Shadow Casting" = Off y "Receive Shadows"= Off

¿Y si tenemos en cuenta los puntos de movimiento?

Veamos cómo introducir un ligero sistema de puntos de movimiento teniendo en cuenta que el personaje se pueda mover solo si tiene disponibles, todavía, los puntos de movimiento necesarios:

```
using UnityEngine;
using UnityEngine.AI;
using System.Collections;
```

```
[RequireComponent(typeof(NavMeshAgent))]
public class ControlMovimientoPorClic : MonoBehaviour
{
    public float puntosMovimientoMaximos = 20f;
    private float puntosMovimientoActuales;

    public LayerMask capaSuelo;
    private NavMeshAgent agente;

    private Coroutine corutinaMovimiento;
    private float distanciaTotalRuta;

    void Start()
    {
        agente = GetComponent<NavMeshAgent>();
        puntosMovimientoActuales = puntosMovimientoMaximos;
    }

    void Update()
    {
        if (Input.GetMouseButtonDown(0) && puntosMovimientoActuales > 0)
        {
            Ray ray = Camera.main.ScreenPointToRay(Input.mousePosition);

            if (Physics.Raycast(ray, out RaycastHit hit, 100f, capaSuelo))
            {
                Vector3 destinoPropuesto = hit.point;

                NavMeshPath ruta = new NavMeshPath();
                if (agente.CalculatePath(destinoPropuesto, ruta))
                {
                    distanciaTotalRuta = CalcularLongitudRuta(ruta); // Almacenamos la distancia total

                    if (distanciaTotalRuta <= puntosMovimientoActuales)
                    {
                        agente.SetPath(ruta);

                        if (corutinaMovimiento != null)
                            StopCoroutine(corutinaMovimiento);

                        corutinaMovimiento = StartCoroutine(ConsumirPuntosDuranteMovimiento());
                    }
                    else
                    {
                        Debug.Log("¡No tienes suficientes puntos para moverte hasta ese lugar!");
                    }
                }
            }
        }
    }
}
```

Recordad tener en NavSurface
preparado para este tipo de
agente!!

```

    }
}

// Recuperar puntos para testeo
if (Input.GetKeyDown(KeyCode.R))
{
    RecuperarPuntosMovimiento();
}

float CalcularLongitudRuta(NavMeshPath ruta)
{
    float total = 0f;
    for (int i = 1; i < ruta.corners.Length; i++)
    {
        total += Vector3.Distance(ruta.corners[i - 1], ruta.corners[i]);
    }
    return total;
}

IEnumerator ConsumirPuntosDuranteMovimiento()
{
    float puntosConsumidos = 0f;
    float progresoAnterior = 0f;

    while (!agente.pathPending && agente.remainingDistance > 0.1f && puntosMovimientoActuales > 0)
    {
        // Calculamos el progreso actual (0 a 1)
        float progresoActual = 1 - (agente.remainingDistance / distanciaTotalRuta);

        // Calculamos cuánto hemos avanzado desde el último frame
        float progresoDelta = progresoActual - progresoAnterior;

        // Calculamos los puntos a consumir en este frame
        float consumoFrame = progresoDelta * distanciaTotalRuta;

        puntosMovimientoActuales -= consumoFrame;
        puntosConsumidos += consumoFrame;
        progresoAnterior = progresoActual;

        // Aseguramos no tener valores negativos
        puntosMovimientoActuales = Mathf.Max(puntosMovimientoActuales, 0f);

        if (puntosMovimientoActuales <= 0f)
        {
            // Detenemos inmediatamente al agente
            agente.ResetPath();
            agente.velocity = Vector3.zero; // Detenemos cualquier movimiento residual
            Debug.Log("¡Te has quedado sin puntos de movimiento!");
            break;
        }

        yield return new WaitForFixedUpdate();
    }

    Debug.Log($"Movimiento terminado. PM restantes: {puntosMovimientoActuales:F2}");
}

```

```
public void RecuperarPuntosMovimiento()
{
    puntosMovimientoActuales = puntosMovimientoMaximos;
    Debug.Log("Puntos de movimiento restaurados.");
}

public float GetPuntosActuales() => puntosMovimientoActuales;
public float GetPuntosMaximos() => puntosMovimientoMaximos;
}
```